

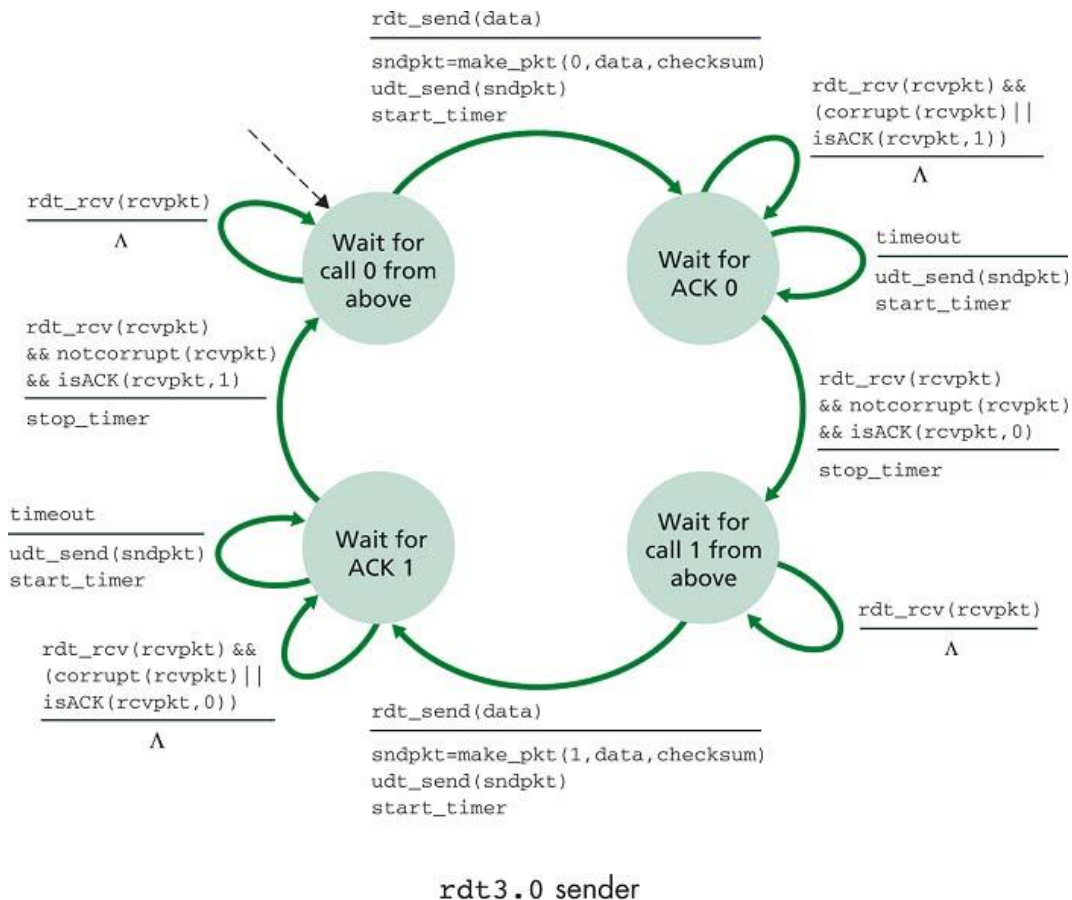
EECE 4830-5830: Network Design

Dr. Vinod Vokkarane

Programming Project Phase 3:

Implement RDT 3.0 over an unreliable UDP channel with bit-errors and loss

Project description: The TCP/IP stack has five layers, namely application, transport, network, link, and physical. In Phases 1-2, each group implemented the standard user datagram protocol (UDP) sockets. The intention was to transfer a file (say JPEG) between a UDP client process and a UDP server process. In Phase 1 and Phase 2, we provided reliable data transfer service over the unreliable bit-error prone UDP connection. In Phase 3, you will have to enhance your sender code to handle two additional scenarios: 1. Data packet loss and 2. ACK packet loss. The FSM of the sender and receiver and shown below (Note: RDT 3.0 Receiver is same as RDT 2.2 Receiver):



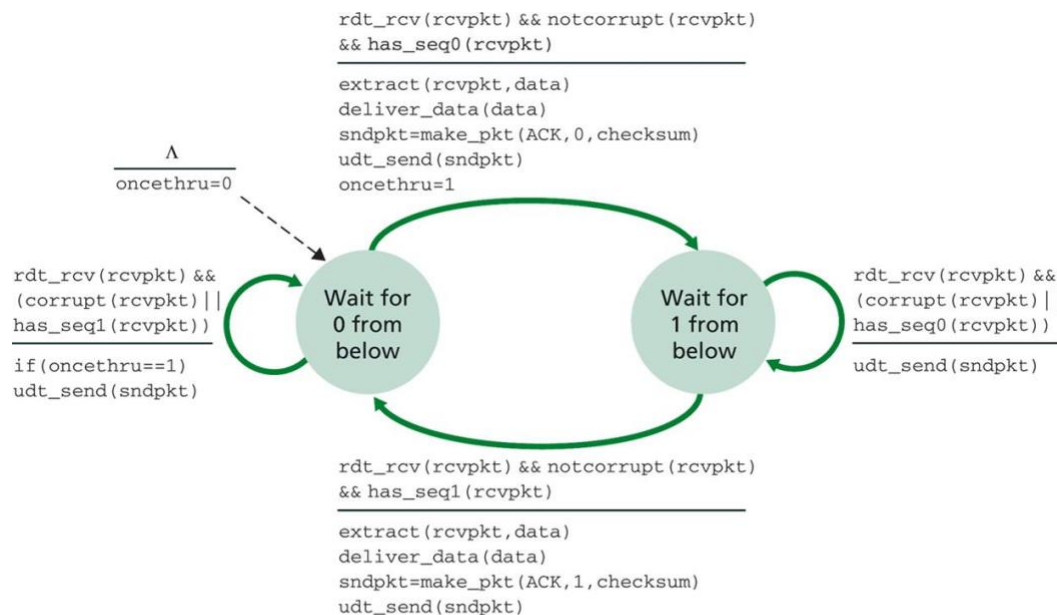


Figure 3.14 ♦ rdt2.2 receiver

The following are the basic implementation steps:

1. Pick a transfer file - JPEG image file (recommended), easier to identify loss of packets in an image file (lost pixels).
2. Make_packet function - parses the image file and breaks it down to several packets (set a fixed packet size, say 1024 bytes).
3. Use the UDP sockets (Phase 1) to send and receive RDT2.2 packets (Phase 2). UDP sockets are unreliable, so they simulate the unreliable Internet IP (network) layer.
4. Implement: Sequence Numbers (to identify duplicates), Checksum (implement your own, similar to UDP), and ACKs (remember, RDT 2.2 is a NAK-free protocol).

New Features for Phase 3:

5. Enhance sender to handle packet loss (RDT 3.0) by using a countdown timer.
6. The RDT3.0 receiver (similar to RDT2.2 receiver) should assemble packets in order and deliver the entire transfer file to the application.

In this phase, you will need to implement different data transfer scenarios

Option 1 - No loss/bit-errors.

Option 2 - ACK packet bit-error: you need to intentionally change the data bits of the received ACK packet at the sender (some error %) and implement suitable recovery mechanism.

Option 3 - Data packet bit-error: you need to intentionally change the data bits of the received DATA packet at the receiver (some error %) and implement suitable recovery mechanism.

Option 4 - ACK packet loss: you need to intentionally drop the received ACK packet at the sender (some loss %) and implement suitable recovery mechanism.

Option 5 - Data packet loss: you need to intentionally drop the received DATA packet at the receiver (some loss %) and implement suitable recovery mechanism.

Expectations: In this phase of the project, you will learn about the basic principles used to provide non-pipelined reliable data transfer over a data channel with bit errors and packet losses.

Deliverables:

1. DESIGN_DOCUMENT.pdf/md

1. If submitting on BlackBoard, it MUST be a PDF
2. If you only want to submit your GitHub repo, it can remain a MD

2. README.pdf/md

3. Plot charts for completion time for all your **5 options** for transferring the same file at **0% loss/error to 95% loss/error in increments of 5%.**

1. Note:

1. Each point on the plot MUST be an average of multiple runs (5) at a given loss/error rate (make sure your file isn't massive, you'll be running for days ☺).
2. For accuracy remove print/debug statements to the output console statements while measuring the completion time for the chart.

4. Individual Contribution (contribution.txt/doc):

This is confidential submission for each member

1. Teammate rating between 1 (not good) – 5 (excellent) on

1. Project time commitment
2. Design contribution
3. Coding contribution
4. Debugging contribution
5. Report preparation

References:

1. Socket Programming
 - [JAVA Socket Programming](#)
 - [Linux Gazette's Socket Programming](#)
 - <http://www.programminglogic.com/sockets-programming-in-c-using-udp-datagrams/>
 - [Socket Programming by JAVA World](#) and [O'REILLY JAVA Network Programming](#)
 - [Python Socket Programming](#)
2. UDP: [RFC768](#)
3. Principles of Reliable Data Transfer, Section 3.4.1, Kurose/Ross (7th Ed).